

Name: Monisha Raghini Chelle

Roll No: 2520030537

Branch: B.Tech CSE

Subject: Linux System Programming Lab

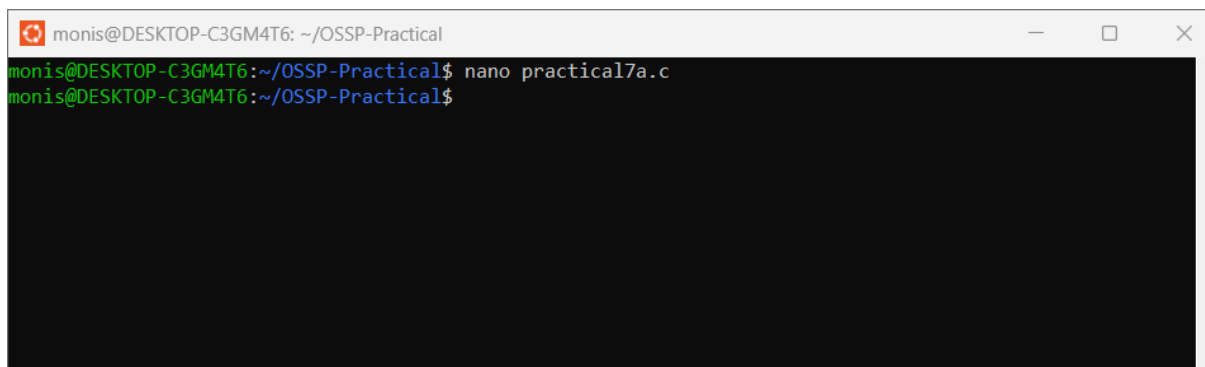
OSSP_Practical7&8

OSSP (Operating Systems And Systems Programming) - 25CS2104E

7. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Step 1 — Create the program

A screenshot of a terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSSP-Practical'. The terminal shows the command 'nano practical7a.c' being entered at the prompt 'monis@DESKTOP-C3GM4T6:~/OSSP-Practical\$'. The prompt is repeated on the next line, indicating the command has been executed and the nano editor is now open. The terminal background is black with green text for the prompt and command.

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical7a.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical7a.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_initialized = 100;
int global_uninitialized;

void display(void)
{
    printf("Function executed successfully.\n");
}

int main(void)
{
    static int static_variable = 50;
    int stack_variable = 25;

    int *heap_variable = malloc(sizeof(int));

    if (heap_variable == NULL) {
        perror("malloc");
        return 1;
    }

    *heap_variable = 75;

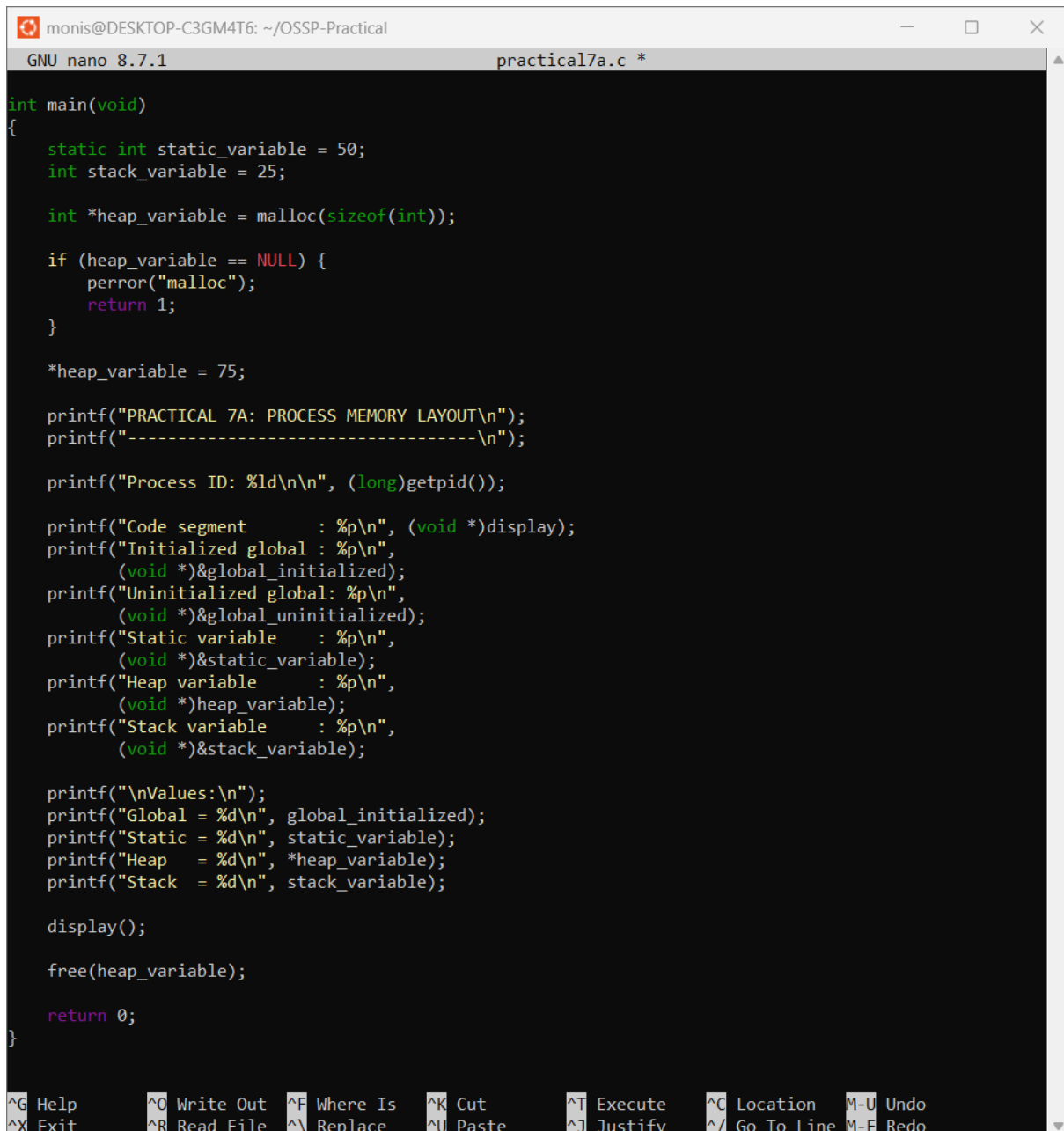
    printf("PRACTICAL 7A: PROCESS MEMORY LAYOUT\n");
    printf("-----\n");

    printf("Process ID: %ld\n\n", (long) getpid());

    printf("Code segment      : %p\n", (void *) display);
    printf("Initialized global : %p\n",
        (void *)&global_initialized);
    printf("Uninitialized global: %p\n",
        (void *)&global_uninitialized);
    printf("Static variable    : %p\n",
        (void *)&static_variable);
    printf("Heap variable      : %p\n",
        (void *) heap_variable);
    printf("Stack variable     : %p\n",
        (void *)&stack_variable);

    printf("\nValues:\n");
    printf("Global = %d\n", global_initialized);
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical7a.c *

int main(void)
{
    static int static_variable = 50;
    int stack_variable = 25;

    int *heap_variable = malloc(sizeof(int));

    if (heap_variable == NULL) {
        perror("malloc");
        return 1;
    }

    *heap_variable = 75;

    printf("PRACTICAL 7A: PROCESS MEMORY LAYOUT\n");
    printf("-----\n");

    printf("Process ID: %ld\n", (long) getpid());

    printf("Code segment      : %p\n", (void *) display);
    printf("Initialized global : %p\n",
        (void *) &global_initialized);
    printf("Uninitialized global: %p\n",
        (void *) &global_uninitialized);
    printf("Static variable   : %p\n",
        (void *) &static_variable);
    printf("Heap variable     : %p\n",
        (void *) heap_variable);
    printf("Stack variable    : %p\n",
        (void *) &stack_variable);

    printf("\nValues:\n");
    printf("Global = %d\n", global_initialized);
    printf("Static = %d\n", static_variable);
    printf("Heap   = %d\n", *heap_variable);
    printf("Stack  = %d\n", stack_variable);

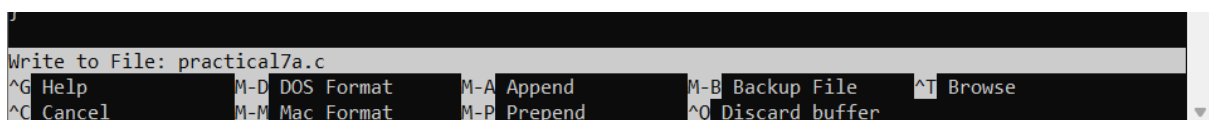
    display();

    free(heap_variable);

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save using Ctrl + O, Enter, then Ctrl + X.



```
Write to File: practical7a.c
^G Help M-D DOS Format M-A Append M-B Backup File ^T Browse
^C Cancel M-M Mac Format M-P Prepend ^O Discard buffer
```

Step 2 — Compile and run

```
gcc -Wall -Wextra practical7a.c -o practical7a
./practical7a
```

```
Select monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical7a.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra practical7a.c -o practical7a
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical7a
PRACTICAL 7A: PROCESS MEMORY LAYOUT
-----
Process ID: 1761

Code segment      : 0x5d4f6608a209
Initialized global : 0x5d4f6608d010
Uninitialized global: 0x5d4f6608d01c
Static variable   : 0x5d4f6608d014
Heap variable     : 0x5d4f7a7f5010
Stack variable    : 0x7ffe7a08bedc

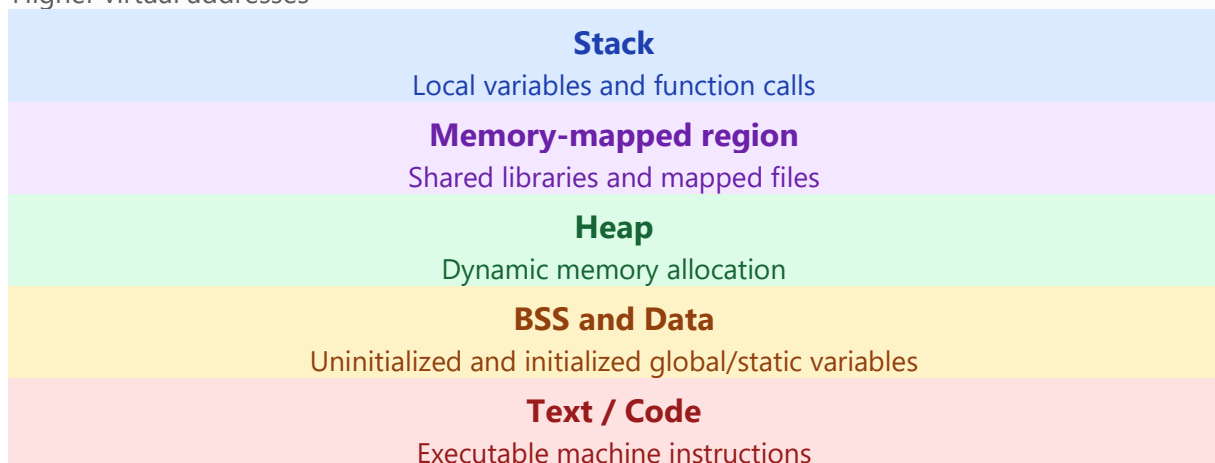
Values:
Global = 100
Static = 50
Heap   = 75
Stack  = 25
Function executed successfully.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 3 — Analyze the memory segments

Typical Linux process memory layout

Conceptual diagram, not to scale. Actual mappings vary.

Higher virtual addresses



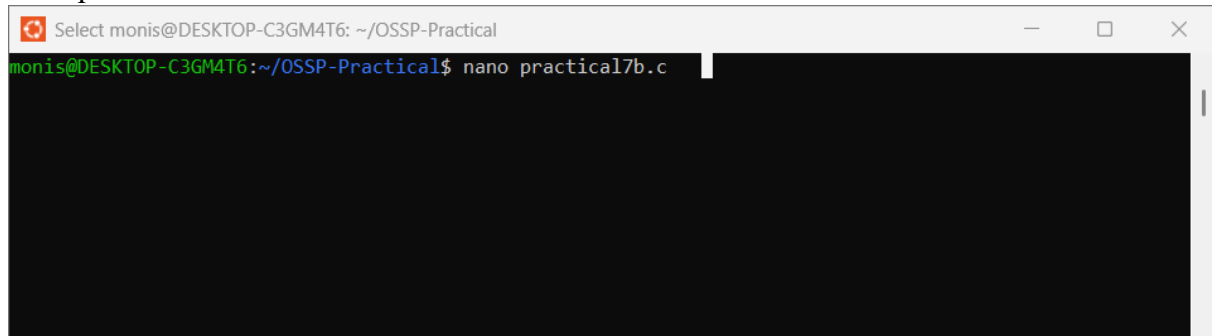
Lower virtual addresses

Segment	Purpose
Text / Code	Contains executable instructions. Usually read-only and executable.
Data	Contains initialized global and static variables.
BSS	Contains uninitialized or zero-initialized global and static variables.

Heap	Stores dynamically allocated memory obtained using functions such as <code>malloc()</code> .
Stack	Stores local variables and function-call information.
Memory-mapped region	Contains shared libraries, mapped files and anonymous mappings.

Step 4 — Create the program

`nano practical7b.c`

A terminal window titled "Select monis@DESKTOP-C3GM4T6: ~/OSSP-Practical" with standard window controls. The prompt is "monis@DESKTOP-C3GM4T6:~/OSSP-Practical\$". The command "nano practical7b.c" has been entered, and a white cursor is visible at the end of the command line. The rest of the terminal area is black and empty.

```
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical7b.c
```



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical7b.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_data = 100;

int main(void)
{
    static int static_data = 200;
    int stack_data = 300;

    char *heap_data = malloc(1024 * 1024);

    if (heap_data == NULL) {
        perror("malloc");
        return 1;
    }

    heap_data[0] = 'A';

    printf("PRACTICAL 7B: VIRTUAL MEMORY MAPPING\n");
    printf("-----\n");

    printf("Process PID: %ld\n", (long)getpid());

    printf("Global address: %p\n", (void *)&global_data);
    printf("Static address: %p\n", (void *)&static_data);
    printf("Heap address : %p\n", (void *)heap_data);
    printf("Stack address : %p\n", (void *)&stack_data);

    printf("\nProcess is running for 60 seconds.\n");
    printf("Examine /proc/%ld/maps now.\n", (long)getpid());

    fflush(stdout);

    sleep(60);

    free(heap_data);

    printf("Memory released. Program completed.\n");

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo

Write to File: practical7b.c
^G Help      M-D DOS Format M-A Append   M-B Backup File ^T Browse
^C Cancel     M-M Mac Format M-P Prepend  ^O Discard buffer
```

Step 5 — Compile and execute

In Terminal 1, run:

```
gcc -Wall -Wextra practical7b.c -o practical7b
./practical7b
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical7b
PRACTICAL 7B: VIRTUAL MEMORY MAPPING
-----
Process PID: 2036
Global address: 0x6169e3137010
Static address: 0x6169e3137014
Heap address : 0x7bec096ff010
Stack address : 0x7ffeab63f96c

Process is running for 60 seconds.
Examine /proc/2036/maps now.
Memory released. Program completed.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 6 — Examine the process memory

Open Terminal 2 and run:

`cd ~/OSSP-Practical`

Replace 2036 below with the actual PID shown in Terminal 1:

`cat /proc/2036/maps`

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical7b
PRACTICAL 7B: VIRTUAL MEMORY MAPPING
-----
Process PID: 2036
Global address: 0x6169e3137010
Static address: 0x6169e3137014
Heap address : 0x7bec096ff010
Stack address : 0x7ffeab63f96c

Process is running for 60 seconds.
Examine /proc/2036/maps now.

monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ cat /proc/2036/maps
0169e313000-0169e3134000 r--p 00000000 08:30 14293 /home/monis/OSSP-Practical/prac
tical7b
0169e3134000-0169e3135000 r-xp 00001000 08:30 14293 /home/monis/OSSP-Practical/prac
tical7b
0169e3135000-0169e3136000 r--p 00002000 08:30 14293 /home/monis/OSSP-Practical/prac
tical7b
0169e3136000-0169e3137000 r--p 00002000 08:30 14293 /home/monis/OSSP-Practical/prac
tical7b
0169e3137000-0169e3138000 rw-p 00003000 08:30 14293 /home/monis/OSSP-Practical/prac
tical7b
0169e3138000-0169e3139000 rw-p 00000000 00:00 0 [heap]
7bec096ff000-7bec09800000 rw-p 00000000 00:00 0 /usr/lib/x86_64-linux-gnu/libc.
so.6
7bec09800000-7bec09820000 r--p 00000000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7bec09820000-7bec099c0000 r-xp 00020000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7bec099c0000-7bec09a0c000 r--p 001c0000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7bec09a0c000-7bec09a12000 r--p 0020c000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7bec09a12000-7bec09a14000 rw-p 00212000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7bec09a14000-7bec09a21000 rw-p 00000000 00:00 0 /usr/lib/x86_64-linux-gnu/ld-11
7bec09a21000-7bec09ab7000 rw-p 00000000 00:00 0 /usr/lib/x86_64-linux-gnu/ld-11
7bec09ab7000-7bec09abf000 rw-p 00000000 00:00 0 /usr/lib/x86_64-linux-gnu/ld-11
7bec09abf000-7bec09ac0000 r--p 00000000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-11
mux-x86_64.so.2
7bec09ac0000-7bec09aef000 r-xp 00001000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-11
mux-x86_64.so.2
7bec09aef000-7bec09afa000 r--p 00030000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-11
mux-x86_64.so.2
7bec09afa000-7bec09afc000 r--p 0003b000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-11
mux-x86_64.so.2
7bec09afc000-7bec09afd000 rw-p 0003d000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-11
mux-x86_64.so.2
7bec09afd000-7bec09afe000 rw-p 00000000 00:00 0 [stack]
7ffeab621000-7ffeab642000 rw-p 00000000 00:00 0 [vvar]
7ffeab736000-7ffeab73a000 r-p 00000000 00:00 0 [vdso]
7ffeab73a000-7ffeab73c000 r-xp 00000000 00:00 0
```

Finish Q7

Run these commands in Ubuntu:

`cd ~/OSSP-Practical`

`./practical7b &`

`PID=$!`

`sleep 1`

`pmap -x "$PID" | tail -8`

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical7b &
[1] 2104
PRACTICAL 7B: VIRTUAL MEMORY MAPPING
-----
Process PID: 2104
Global address: 0x58f6c73fd010
Static address: 0x58f6c73fd014
Heap address : 0x73c0c58ff010
Stack address : 0x7ffeffaaa91c

Process is running for 60 seconds.
Examine /proc/2104/maps now.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ PID=$!
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ sleep 1
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ pmap -x "$PID" | tail -8
000073c0c5d3d000      8      8      8 r---- ld-linux-x86-64.so.2
000073c0c5d3f000      4      4      4 rw--- ld-linux-x86-64.so.2
000073c0c5d40000      4      4      4 rw--- [ anon ]
00007ffeffa8c000    132    12    12 rw--- [ stack ]
00007ffeffbe0000     16      0      0 r---- [ anon ]
00007ffeffbe4000      8      4      0 r-x-- [ anon ]
-----
total kB          3788    1920    100
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ cat > practical7_report.txt
Memory released. Program completed.
```

Create the report quickly:

```
cat > practical7_report.txt <<'EOF'
```

PRACTICAL 7 - MEMORY ANALYSIS

The code segment contains executable instructions.

The data segment stores initialized global and static variables.

The BSS segment stores uninitialized global and static variables.

The heap stores dynamically allocated memory.

The stack stores local variables and function-call information.

The /proc/PID/maps file shows virtual address ranges,
memory permissions, mapped files, the heap and the stack.

The pmap tool displays the memory mappings and their sizes.
ASLR changes memory addresses between executions.

RESULT:

Linux process memory organization and virtual memory
mappings were successfully examined.

EOF

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ cat > practical7_report.txt <<'EOF'
> PRACTICAL 7 - MEMORY ANALYSIS

The code segment contains executable instructions.
The data segment stores initialized global and static variables.
The BSS segment stores uninitialized global and static variables.
The heap stores dynamically allocated memory.
The stack stores local variables and function-call information.

The /proc/PID/maps file shows virtual address ranges,
memory permissions, mapped files, the heap and the stack.

The pmap tool displays the memory mappings and their sizes.
ASLR changes memory addresses between executions.

RESULT:
Linux process memory organization and virtual memory
mappings were successfully examined.
EOF
```

One final check: Run this command to confirm that your report was saved:
cat practical7_report.txt

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ cat > practical7_report.txt <<'EOF'
> PRACTICAL 7 - MEMORY ANALYSIS

The code segment contains executable instructions.
The data segment stores initialized global and static variables.
The BSS segment stores uninitialized global and static variables.
The heap stores dynamically allocated memory.
The stack stores local variables and function-call information.

The /proc/PID/maps file shows virtual address ranges,
memory permissions, mapped files, the heap and the stack.

The pmap tool displays the memory mappings and their sizes.
ASLR changes memory addresses between executions.

RESULT:
Linux process memory organization and virtual memory
mappings were successfully examined.
EOF
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ cat practical7_report.txt
PRACTICAL 7 - MEMORY ANALYSIS

The code segment contains executable instructions.
The data segment stores initialized global and static variables.
The BSS segment stores uninitialized global and static variables.
The heap stores dynamically allocated memory.
The stack stores local variables and function-call information.

The /proc/PID/maps file shows virtual address ranges,
memory permissions, mapped files, the heap and the stack.

The pmap tool displays the memory mappings and their sizes.
ASLR changes memory addresses between executions.

RESULT:
Linux process memory organization and virtual memory
mappings were successfully examined.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

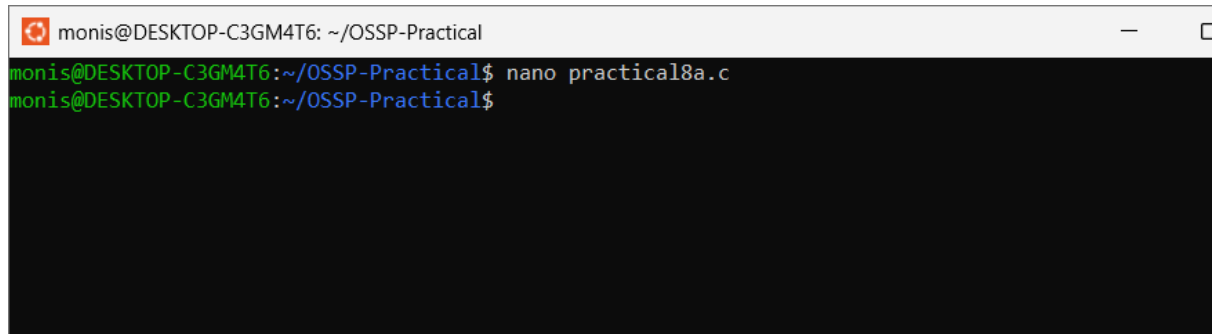
8. Develop a program using `malloc()`, `calloc()`, `realloc()`, and `free()`. Monitor memory allocation behavior and identify memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

Step 1 — Create `practical8a.c`

```
cd ~/OSSP-Practical
```

```
nano practical8a.c
```

A terminal window with a title bar that reads "monis@DESKTOP-C3GM4T6: ~/OSSP-Practical". The terminal shows two lines of command history: "monis@DESKTOP-C3GM4T6:~/OSSP-Practical\$ nano practical8a.c" and "monis@DESKTOP-C3GM4T6:~/OSSP-Practical\$". The terminal background is black, and the text is green. The cursor is at the end of the second line.

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical8a.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical8a.c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *a, *b;

    /* malloc */
    a = malloc(5 * sizeof(int));

    if (a == NULL) {
        perror("malloc");
        return 1;
    }

    printf("malloc values: ");
    for (int i = 0; i < 5; i++) {
        a[i] = (i + 1) * 10;
        printf("%d ", a[i]);
    }

    /* calloc */
    b = calloc(5, sizeof(int));

    if (b == NULL) {
        perror("calloc");
        free(a);
        return 1;
    }

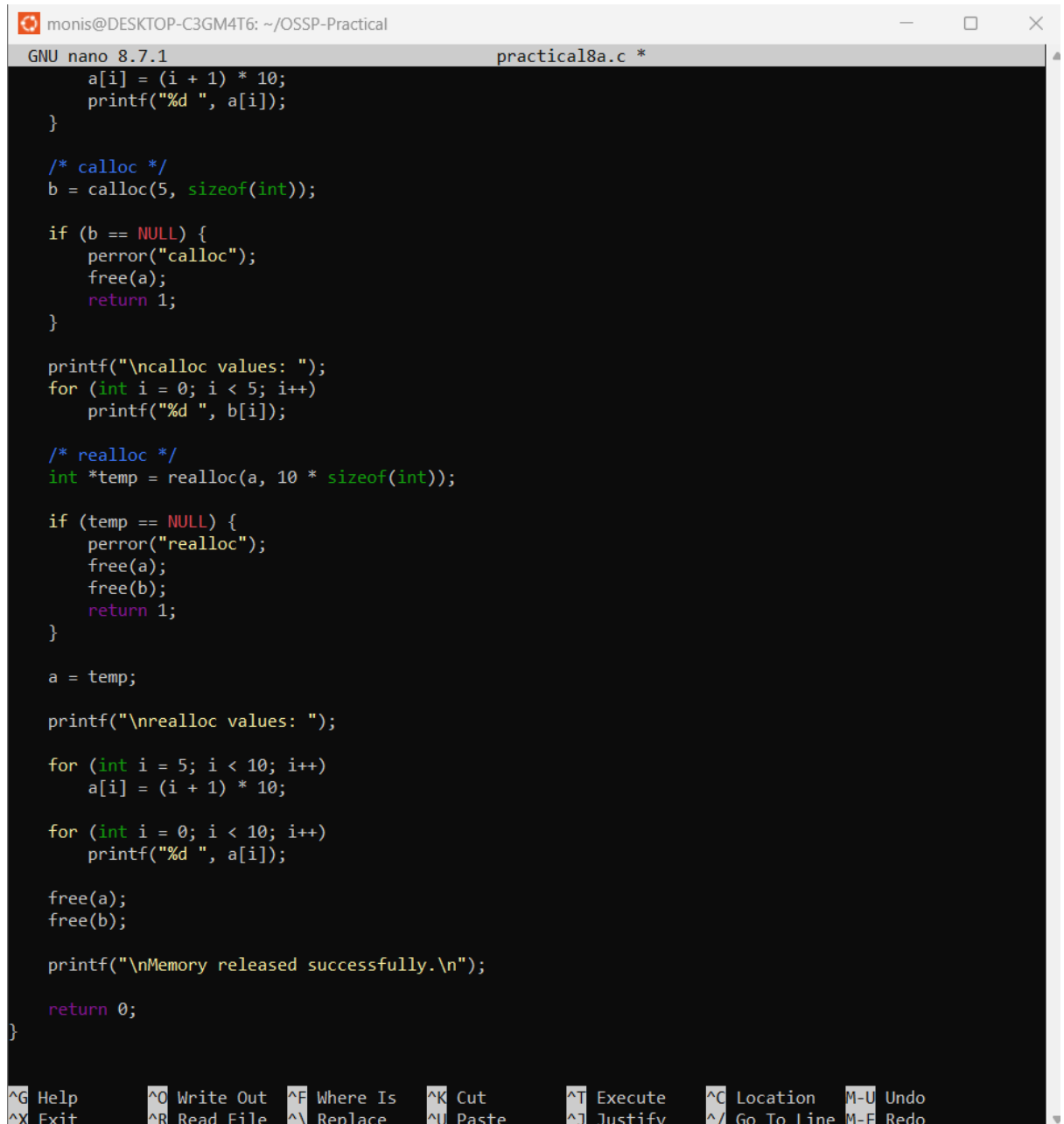
    printf("\ncalloc values: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", b[i]);

    /* realloc */
    int *temp = realloc(a, 10 * sizeof(int));

    if (temp == NULL) {
        perror("realloc");
        free(a);
        free(b);
        return 1;
    }

    a = temp;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^H Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical8a.c
    a[i] = (i + 1) * 10;
    printf("%d ", a[i]);
}

/* calloc */
b = calloc(5, sizeof(int));

if (b == NULL) {
    perror("calloc");
    free(a);
    return 1;
}

printf("\ncalloc values: ");
for (int i = 0; i < 5; i++)
    printf("%d ", b[i]);

/* realloc */
int *temp = realloc(a, 10 * sizeof(int));

if (temp == NULL) {
    perror("realloc");
    free(a);
    free(b);
    return 1;
}

a = temp;

printf("\nrealloc values: ");

for (int i = 5; i < 10; i++)
    a[i] = (i + 1) * 10;

for (int i = 0; i < 10; i++)
    printf("%d ", a[i]);

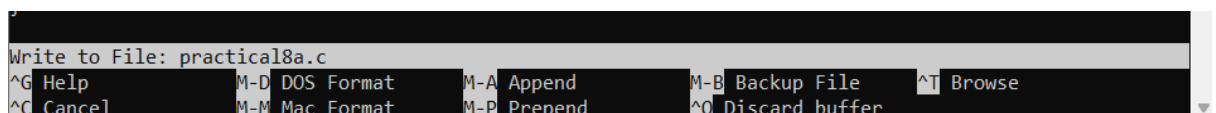
free(a);
free(b);

printf("\nMemory released successfully.\n");

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_/ Go To Line M-E Redo

Save with Ctrl + O, Enter, then Ctrl + X.



```
Write to File: practical8a.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

Step 2 — Compile and run

```
gcc -g -Wall -Wextra practical8a.c -o practical8a
./practical8a
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical8a.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -g -Wall -Wextra practical8a.c -o practical8a
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical8a
malloc values: 10 20 30 40 50
calloc values: 0 0 0 0 0
realloc values: 10 20 30 40 50 60 70 80 90 100
Memory released successfully.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 3 — Check memory leaks

valgrind --leak-check=full ./practical8a

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ valgrind --leak-check=full ./practical8a
==2249== Memcheck, a memory error detector
==2249== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2249== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2249== Command: ./practical8a
==2249==
malloc values: 10 20 30 40 50
calloc values: 0 0 0 0 0
realloc values: 10 20 30 40 50 60 70 80 90 100
Memory released successfully.
==2249==
==2249== HEAP SUMMARY:
==2249==    in use at exit: 0 bytes in 0 blocks
==2249==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==2249==
==2249== All heap blocks were freed -- no leaks are possible
==2249==
==2249== For lists of detected and suppressed errors, rerun with: -s
==2249== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 4 — Create practical8b.c

nano practical8b.c

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical8b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical8b.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define SIZE (20 * 1024 * 1024)

void show_memory(const char *label)
{
    char path[100];
    char line[256];

    snprintf(path, sizeof(path),
             "/proc/%ld/status", (long)getpid());

    FILE *fp = fopen(path, "r");

    if (fp == NULL) {
        perror("fopen");
        return;
    }

    printf("\n%s (PID=%ld)\n", label, (long)getpid());

    while (fgets(line, sizeof(line), fp)) {
        if (strncmp(line, "VmRSS:", 6) == 0 ||
            strncmp(line, "VmSize:", 7) == 0)
            printf("%s", line);
    }

    fclose(fp);
}

int main(void)
{
    char *data = malloc(SIZE);

    if (data == NULL) {
        perror("malloc");
        return 1;
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical8b.c *
}

/* Touch each page to allocate physical memory. */
for (size_t i = 0; i < SIZE; i += 4096)
    data[i] = 'A';

printf("PRACTICAL 8B: COPY-ON-WRITE\n");

show_memory("Parent before fork");

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    free(data);
    return 1;
}

if (pid == 0) {
    show_memory("Child before modification");

    /* Modify every page to trigger Copy-on-Write. */
    for (size_t i = 0; i < SIZE; i += 4096)
        data[i] = 'B';

    show_memory("Child after modification");

    printf("\nChild data[0] = %c\n", data[0]);

    fflush(stdout);
    free(data);
    _exit(0);
}

waitpid(pid, NULL, 0);

printf("\nParent data[0] = %c\n", data[0]);

show_memory("Parent after child modification");

free(data);

return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-F Redo

Write to File: practical8b.c
^G Help      M-D DOS Format M-A Append   M-B Backup File ^T Browse
^C Cancel     M-M Mac Format M-P Prepend  ^O Discard buffer
```

Step 5 — Compile and run

```
gcc -g -Wall -Wextra practical8b.c -o practical8b
./practical8b
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical8b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -g -Wall -Wextra practical8b.c -o practical8b
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical8b
PRACTICAL 8B: COPY-ON-WRITE

Parent before fork (PID=2287)
VmSize: 23244 kB
VmRSS: 22144 kB

Child before modification (PID=2288)
VmSize: 23244 kB
VmRSS: 21388 kB

Child after modification (PID=2288)
VmSize: 23244 kB
VmRSS: 21388 kB

Child data[0] = B
Parent data[0] = A

Parent after child modification (PID=2287)
VmSize: 23244 kB
VmRSS: 22144 kB
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```